MIT 8103 Advanced Database Systems: Portfolio Summary

Case Study: Clinic/Hospital System

Name: Bolurin Joshua Alao

Student ID: 301904150

Repository: https://github.com/bolualao/Alao_Bolurin_301904150

Case study

The project involved designing and implementing a clinic?Hospital management system database to track patients, doctors, medical appointments, and consultation records. The system is designed to allow clinic receptionists and doctors to manage daily schedules, record clinical diagnoses, and maintain accurate patient histories while preventing scheduling conflicts.

Portfolio 1: Database Design and Modelling

To establish the foundational system, a relational schema was designed and deployed using PostgreSQL in a GitHub Codespaces environment. Four tables were created: `department`, `patient`, `doctor`, and `appointment`. Business rules were enforced at the database level using primary keys, foreign key constraints, `NOT NULL` constraints, and a `CHECK` constraint to prevent consultation notes from predating appointments. Partial unique indexes were implemented to actively prevent doctors and patients from being double-booked.

The implemented schema was verified using a script containing both valid data and deliberate constraint violations. The database successfully blocked all invalid inserts, including missing patient names, duplicate medical license numbers, and overlapping active appointments. A final count query verified that only the original, valid sample records (2 departments, 2 patients, 2 doctors, and 4 appointments) were written to the database, confirming the schema successfully guarantees data integrity.

Portfolio 2: Query Processing and Optimisation

This stage explored data retrieval and query performance tuning. SQL queries were written to filter specific patient appointment histories, calculate doctors' total workload using `LEFT JOIN` aggregations, and identify doctor availability using a `NOT EXISTS` clause. To test performance, a synthetic `appointment_history` table containing 100,000 rows was generated. A query filtering by `patient_id` and `scheduled_at` was benchmarked.

A composite B-tree index was then created on these two columns. The execution plans and times were recorded and compared:

Metric	Before Indexing	After Indexing
Scan Type	Seq Scan	Bitmap Heap Scan
Median Time	7.133 ms	1.723 ms
Shared Buffer Hits	1,235	995

The index successfully optimized the query by changing the execution plan from scanning all 100,000 rows to a Bitmap Heap Scan, which looks up qualifying blocks in the index before retrieving the necessary heap data. This reduced memory buffer reads and lowered execution time significantly (PostgreSQL Global Development

Group, 2025a). The measured times represent a local snapshot; real-world execution times fluctuate based on active server load, and maintaining this index will introduce minor write-overhead on future insertions.

Portfolio 3: Transactions and Concurrency

To ensure data consistency during complex updates and simultaneous user activity, PostgreSQL transactions and concurrency controls were tested. A valid transaction block correctly updated a scheduled appointment and committed the changes permanently. A second transaction attempted to update a patient's phone number but was followed by an invalid insertion that triggered a constraint error. Issuing a `ROLLBACK` successfully reverted the database to its earlier state, discarding the phone number change to prevent a partial, corrupted update.

Concurrency was tested using two simultaneous database sessions operating under the `read committed` isolation level. Session A initiated an insert to book a time slot but left the transaction open. When Session B attempted to book the exact same slot, the database recognized the uncommitted lock and forced Session B to wait. Once Session A finalized its changes with a `COMMIT`, Session B immediately failed with a duplicate key violation. This verified that the database successfully isolates concurrent operations and prevents race conditions from bypassing scheduling constraints.

Portfolio 4: NoSQL and Advanced Data Models

To evaluate alternative database paradigms, a document-oriented data model was implemented using MongoDB. Instead of separating entities into distinct relational tables, an event-based `clinic_events` collection was created. This schema accommodates polymorphism, allowing a single collection to store different event types (like scheduling an appointment versus recording consultation notes) natively. Core details were shared across documents, while specific patient and doctor details were embedded to avoid complex table joins.

The collection was tested using a local MongoDB community server. Four varying event documents were inserted successfully. Queries verified the ability to filter documents based on deeply nested fields (such as `doctor.department`) and sort embedded patient histories in chronological order. While MongoDB proved highly flexible for event logging, embedding data means that if a doctor changes departments in the future, the system must locate and update every historical event document they appear in, creating data duplication challenges not present in a normalized SQL model.

Portfolio 5: Distributed and Cloud Database Exercise

The current database architecture consists of a standalone local MongoDB instance and a local PostgreSQL server running within a single GitHub Codespace. To achieve high availability for the clinic, I proposed a theoretical distributed architecture consisting of a MongoDB replica set with three voting nodes placed in separate availability zones. Note: This is a proposed deployment design; no cloud clusters were actively instantiated or tested for this portfolio.

In the proposed design, read and write operations are routed to the Primary node, with Secondaries asynchronously replicating the data to maintain consistency (MongoDB, 2024a). If the Primary node fails, the surviving nodes hold an automatic election to promote a new Primary, minimizing downtime (MongoDB, 2024b). Employing a "majority" write concern ensures data is safely distributed across the cluster before acknowledging success (MongoDB, 2024c). While deploying this to a managed cloud environment like MongoDB Atlas provides automated backups and high security, it incurs ongoing costs and does not replicate the relational PostgreSQL tables, meaning both distinct systems must be managed separately.

Evidence

- Schema Definition - (https://github.com/bolualao/Alao_Bolurin_301904150/blob/main/portfolio-1-database-design/Alao_Bolurin_301904150_P1_Schema.sql)

- Constraint Tests (https://github.com/bolualao/Alao_Bolurin_301904150/blob/main/portfolio-1-database-design/Alao_Bolurin_301904150_P1_TestSQL)

- Optimisation Benchmark - (https://github.com/bolualao/Alao_Joshua_301904150/blob/main/portfolio-2-query-optimisation/Alao_Bolurin_301904150_P2_Optimisation.sql)

- Transactions Script - (https://github.com/bolualao/Alao_Joshua_301904150/blob/main/portfolio-3-transactions-concurrency/Alao_Bolurin_301904150_P3_Transactions.sql)

- MongoDB Queries - (https://github.com/bolualao/Alao_Joshua_301904150/blob/main/portfolio-4-nosql/Alao_Bolurin_301904150_P4_MongoDB.js)

- Screenshots/Evidence Folder - (https://github.com/bolualao/Alao_Bolurin_301904150/tree/main/portfolio-1-database-design/screenshots)

Individual reflection

Design choice: My database design choices were solely based on the Case Study/Organisation Type, Possible Database Operations, the Nature of the Database, and the expected deliverables provided in the MIT 8103 Advanced Database Systems continuous assessment guidelines.

Difficulty: The major difficulty I faced was navigating the GitHub Codespace environment and writing the correct queries to perform each task exactly as required.

Action: The project required intensive research to first fully understand the requirements and requests, and then to carefully aim at answering all questions asked.

Lesson: Database Management is a vital skill to every organization that utilizes data and is seeking to optimize data operations within the organization.

References

MongoDB Manual: Replication. Available at: https://www.mongodb.com/docs/manual/replication/.

PostgreSQL Global Development Group (2025a) PostgreSQL 17.2 Documentation: 14.1. Using EXPLAIN. Available at: https://www.postgresql.org/docs/current/using-explain.html

PostgreSQL Global Development Group (2025b) PostgreSQL 17.2 Documentation: 13.2. Transaction Isolation. Available at: https://www.postgresql.org/docs/current/transaction-iso.html (Accessed: 14 January 2025).

AI-use declaration

I used two AI tools during this work: Gemini and Claude. They were used for the following purposes: correcting queries, rewriting and rephrasing sentences, rectifying connection issues, and providing appropriate advice where needed.

My verification: I personally reviewed the generated queries, ran them in my Codespaces environment to verify their functionality and results, and reviewed all rephrased texts to ensure they accurately reflected my actual work and design intentions.